

AI–Collective: A Ports-and-Adapters Core for Configurable Multi-Agent LLM Orchestration

SuZeAI (SuzeNith)

Independent Researcher

suzeai545@gmail.com

<https://github.com/SuZeAI/ai-collective>

Abstract—We describe the core of AI–Collective, an open, source-available system for running large language model (LLM) agents individually or as coordinated multi-agent systems. The core is organized as a ports-and-adapters (hexagonal) codebase: a framework-free domain layer defines the orchestration logic, an application layer exposes it through provider-agnostic Protocols, and an infrastructure layer supplies concrete adapters for LLM providers, storage, task queues, and distributed locks. On top of this, six LangGraph-based orchestration topologies (sequential, ring, supervisor, tree, mesh, and a user-defined custom graph) share a common runtime that bounds recursion, tolerates partial failure, and absorbs transient provider errors. A three-tier memory subsystem (per-turn context, a per-conversation knowledge graph, and a scope-isolated long-term memory store) and a sixteen-stage LLM middleware stack (loop detection, retries, prompt caching, cost budgeting, guardrails) are applied uniformly to every agent call. We present the core’s layering, its orchestration and memory internals, its middleware composition strategy, and its security controls (SSRF protection, JWT hardening, sandboxed tool execution), and discuss the design trade-offs that follow from targeting deployments ranging from a single laptop to a distributed Kubernetes cluster from one codebase.

Index Terms—multi-agent systems, large language models, LLM orchestration, LangGraph, hexagonal architecture, agent memory, tool use, core architecture

I. INTRODUCTION

Serving an LLM as a multi-turn, tool-using agent – and, further, as a member of a coordinated group of agents – places different demands on a system than serving a stateless completion endpoint. The core must choose a communication topology among agents, bound and recover from long-running or failing model/tool calls, manage memory at more than one time scale, and remain portable across LLM providers and infrastructure tiers without becoming entangled with any one of them.

This paper documents the *core* of AI–Collective: the component that implements this functionality. We deliberately scope the paper to that core: the orchestration runtime, memory subsystem, middleware stack, and the ports-and-adapters layering that keeps them independent of any concrete LLM vendor or infrastructure choice. Client-facing concerns are out of scope.

The remainder of the paper is organized as follows. Section II situates the work relative to existing multi-agent frameworks. Section III describes the core’s layered architecture. Section IV details the multi-agent orchestration runtime. Section V describes the memory subsystem. Section VI describes the

LLM middleware stack. Section VII covers tool execution and sandboxing. Section VIII covers storage/queue/lock elasticity. Section IX covers the security model. Section X discusses limitations, and Section XI concludes.

II. RELATED WORK

AutoGen [2] generalizes single-user/assistant chat into programmable conversation among configurable agents. MetaGPT [3] assigns agents fixed software-engineering roles that communicate through structured document artifacts. CrewAI [4] offers a process-oriented “crew of agents with roles and tasks” abstraction. LangGraph [1], on which this core’s execution layer is built, provides a lower-level primitive: agents and control flow are represented explicitly as nodes and edges in a state graph, with first-class persistence, streaming, and human-in-the-loop interrupts.

The AI–Collective core sits above LangGraph and contributes what these frameworks typically leave to the integrator: (1) a fixed catalogue of *selectable* topologies for common collaboration patterns, with an explicit escape hatch to a fully custom graph; (2) a hexagonal layering that keeps orchestration logic independent of the concrete LLM provider, persistence engine, task queue, and lock implementation; and (3) a uniformly applied middleware stack that turns reliability and cost controls (loop detection, retries, budgets) into cross-cutting, declaratively configured behavior rather than per-agent boilerplate.

III. CORE ARCHITECTURE

The core follows a ports-and-adapters (hexagonal) layering with dependencies pointing strictly inward:

Package	Responsibility
api/	FastAPI app, routers, schemas, DI, auth
app/	Use-case services; <code>ports/</code> holds Protocols
domain/	Framework-free business logic, dataclass models
infra/	Adapters implementing <code>app.ports</code>
share/	Small cross-layer utilities (logging, text)

`domain` imports nothing from `app`, `api`, or `infra`. `app` depends only on Protocols declared in `app/ports/` (`repositories.py`, `llm.py`, `staff_graph.py`, `security.py`) and never on a concrete adapter. `infra` supplies the implementations – JSON-file or MongoDB repositories, LangChain-based LLM provider adapters, RabbitMQ or in-memory task queues, Redis or threading

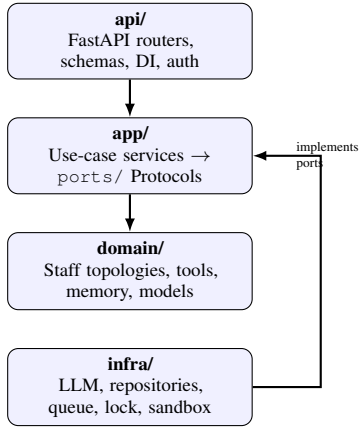


Fig. 1. Core layering. domain depends on nothing outward; app depends only on ports Protocols; infra implements those Protocols and is wired in by api/deps.py.

locks, local or Kubernetes sandboxes. `api/deps.py` is the composition root: it wires concrete adapters into application services through FastAPI’s dependency-injection system. Domain-level errors are translated to HTTP status codes by exception handlers registered centrally, so neither the domain nor the application layer constructs an HTTP response directly.

A request is handled in four steps: (1) FastAPI routes it to a handler in `api/routers/`; (2) a `Depends(...)` factory builds the relevant service with its injected repositories and LLM provider; (3) the service runs the use case against domain logic and ports; (4) domain errors are mapped to HTTP responses by handlers in `api/main.py`. This strict separation is what allows Sections IV through VIII to describe orchestration, memory, and infrastructure choice independently: none of them can leak into the others through an import.

IV. MULTI-AGENT ORCHESTRATION RUNTIME

Multi-agent execution is implemented in `domain/staff/` as six LangGraph state machines, selected at request time via `api/deps.get_staff_graph_service(mode=...)`. Each topology exposes a blocking `run(...)` and a streaming `run_stream(...)`, both accepting the user input, the participating agent definitions, an `LLMProvider`, a round bound, and an optional knowledge-graph context provider.

Mode	Behavior
sequential	One agent processes the request in a single pass (default).
ring	Agents take turns in a fixed circular order until a round bound; each turn sees full history.
supervisor	A lead agent delegates to workers via an inline control tag and synthesizes their replies.
tree	Agents are arranged as a parent/child tree; results roll up leaves-to-root.
mesh	A hub agent selects among N spokes via control tags (<code><NEXT_AGENT></code> , <code><DISCUSSION_END></code>).
custom	A user-defined LangGraph DAG (<code>CustomGraphSpec</code>).

A. Shared runtime guarantees

All six topologies build on `_graph_runtime.py`, which supplies three properties:

- **Bounded recursion.** `recursion_config(max_rounds)` sets the LangGraph recursion limit to $\max(25, 2 \cdot \text{max_rounds} + 10)$, so legitimately long runs do not trip the default 25-superstep ceiling while worst-case execution remains bounded.
- **Graceful degradation.** `run_to_final_state` streams the graph in values mode and, if the recursion limit is hit or a node raises, returns the *latest partial state* instead of discarding the run.
- **Resilient model calls.** `safe_chat` wraps every `llm.chat` call with a bounded timeout and retry/backoff; on exhaustion it substitutes an error string for that turn rather than aborting the run.

A further invariant holds across all six implementations: LangGraph state is never mutated in place. Extending a collection in state (e.g. `conversation_history`) requires building a new container, which LangGraph’s checkpointing and state-diffing depend on.

B. Token budgeting and subagents

`token_budget.py` trims per-turn context to `staff.context_token_limit` minus `staff.output_token_reserve`; this estimate covers system- and user-authored text only and does not account for bound tool-schema tokens. Beyond the six topologies, an individual agent’s tool loop may spawn *subagents* (`subagents.py`) bounded by `subagent_max_concurrent` and `subagent_max_turns` for parallel task delegation; subagents may be granted sandboxed tools, with the sandbox (Section VII) as the enforced security boundary.

V. MEMORY SUBSYSTEM

The core separates memory into three tiers of increasing durability.

A. Working memory and knowledge graph

Within one conversation, turns are ingested into a knowledge graph (`domain/memory/`) that extracts entities and relationships using either a rule-based static extractor (`spaCy`) or an `llm`-based extractor, selected by `graph.build_mode`. This graph is scoped to a single `conversation_id`.

B. Long-term memory

Long-term memory (LTM) persists facts, preferences, and episodic notes *across* tasks. It is off by default, in which case the core behaves as if it did not exist. Each record is scoped along three independent dimensions – `company_id`, `owner_id`, `staff_id` – any of which may be unset (“applies broadly”); a recall query matches records that are equal-or-broader on every dimension, so isolation between users and agents holds regardless of scope breadth. Isolation is enforced both client-side (brute-force/FAISS scoring) and server-side (a Qdrant

TABLE I
DEFAULT MIDDLEWARE STACK, IN COMPOSITION ORDER.

#	Middleware	Hook	Default
1	ModelCallLimit	model	on
2	ToolCallLimit	tool	off
3	Guardrail	tool	off
4	ToolResultCache	tool	off
5	LoopDetection	tool	on
6	ToolRetry	tool	on (2×)
7	ToolTimeout	tool	on
8	PIIRedaction	tool	off
9	AnthropicPromptCaching	model	off (Anthropic only)
10	ModelFallback	model	off
11	ModelRetry	model	off
12	ContextEditing	model	off
13	Summarization	model	off
14	RollingSummary	model	off
15	LongTermMemory	model	off
16	CostBudget	model	off

payload filter), so the guarantee does not depend on which vector-store adapter is active.

Two operations drive LTM, both implemented as LLM middleware (Section VI): **recall** injects a scoped digest as a system message before every model call, and **consolidation**, on a clean run finish, promotes salient short-term notes into LTM with near-duplicate deduplication. Recall scoring blends semantic similarity (dense cosine, when embeddings are enabled) with importance and recency, falling back to lexical term overlap otherwise. The embedding provider is itself pluggable (`infra/llm/embeddings.py`) – a dependency-free hashing embedder by default, or Google/OpenAI/OpenAI-compatible embeddings – mirroring the LLM provider abstraction. An optional approximate-nearest-neighbor index (FAISS locally, Qdrant as an external service, both under `infra/vector_store/`) replaces the default brute-force cosine scan at scale; if the index is unavailable or returns nothing, recall transparently falls back to brute force.

VI. LLM MIDDLEWARE STACK

The LLM provider’s `chat()` (`infra/llm/`) delegates to LangChain’s `create_agent` rather than a hand-rolled ReAct loop, and every cross-cutting concern is expressed as composable middleware, assembled by `build_default_middleware()` (`infra/llm/middleware/builder.py`) and applied to every agent in every topology. Each middleware hooks either tool execution (`awrap_tool_call`) or the model call (`before_model/after_model`).

The order (Table I) is deliberate: guardrail and PII-redaction protect tool execution before it happens; the result cache is checked before retry/timeout do real work; loop-detection, retry, and timeout wrap the tool call itself; and the model-facing group – trim history, recall long-term memory, enforce a cost budget – acts around the model call. **LoopDetection**, on by default, detects an agent re-issuing an identical tool call (same name and arguments) a configurable number of times and nudges it to change approach or finalize, without hard-aborting the

run. **AnthropicPromptCaching** marks the system prompt, tool definitions, and last-message prefix as cacheable so repeated calls sharing that prefix – agent-loop rounds, ring/sequential hops, subagent fan-out – are served from provider-side cache; it is a strict prefix match, so any earlier byte-level change invalidates the cache for everything after it, and it is a silent no-op on non-Anthropic providers.

VII. TOOL EXECUTION AND SANDBOXING

Capabilities beyond text generation are exposed as tool implementations in `domain/tools/`, bound per-agent rather than hard-coded into a role. Code-execution tools (Bash, Python) run through a `SandboxProvider` (`infra/sandbox/`) with two modes: a local subprocess on the host machine, suited to development, or a per-request Kubernetes pod spawned by a dedicated provisioner service for production isolation with automatic lifecycle management. Outbound-request tools (HTTP client, browser navigation) are routed through an SSRF guard (Section IX) rather than trusting tool-supplied URLs directly.

VIII. STORAGE, QUEUEING, AND LOCKING ELASTICITY

Three infrastructure concerns are each abstracted behind an `app/ports` Protocol and selected purely by configuration:

- **Storage** (`infra/repositories/`) – JSON files loaded into memory and rewritten on update (default; adequate for small/medium datasets), or MongoDB via the async Motor driver.
- **Task queue** (`infra/task_queue.py`) – an in-process `ThreadPoolExecutor` by default, or RabbitMQ to detach long-running runs onto distributed workers.
- **Distributed lock** (`infra/lock_provider.py`) – a threading lock by default, or Redis when staff-graph transitions must be mutually excluded across multiple concurrently running processes.

This three-axis elasticity, combined with the local/Kubernetes sandbox choice (Section VII), lets one deployable image serve both a single-laptop evaluation deployment and a horizontally scaled, multi-tenant production deployment without a code change.

IX. SECURITY MODEL

Because bound tools can make outbound network requests, execute code, and act on third-party accounts, several controls are enforced at the framework level rather than left to individual tool implementations.

Authentication. Passwords are hashed with `bcrypt` (`api/security.py`); sessions use HS256-signed JWT access tokens validated on every request. In production (`ENVIRONMENT=production`), startup fails fast if the JWT signing secret is the placeholder default or shorter than 32 bytes.

SSRF protection. Any tool capable of fetching an arbitrary URL is routed through a guard (`domain/tools/_ssrf.py`) that restricts schemes to `http/https`, resolves the hostname and rejects addresses that are private, loopback, link-local, reserved, multicast, or

unspecified – blocking cloud metadata endpoints and internal services – and re-validates the target of every redirect to prevent redirect-based SSRF bypass.

Webhook verification. Inbound platform webhooks (domain/third_party/) are authenticated per-platform before any processing; XML ingestion is parsed with `defusedxml` to prevent XXE and billion-laugh payloads.

Secret hygiene. Sensitive dataclass fields (password hashes, OAuth provider identifiers) are excluded from Python’s automatic `repr`, so a logged object or unhandled traceback cannot leak a credential.

X. LIMITATIONS AND FUTURE WORK

Token budgeting accounts only for system- and user-authored text, not for the schema size of bound tools, which can understate context pressure for agents with many tools attached. Delegation in the supervisor, tree, and mesh topologies relies on inline text control tags rather than a structured, provider-native function-call channel, trading provider portability for text-parsing robustness. The default JSON storage adapter loads each collection into memory and rewrites it on update, adequate for small/medium datasets but not intended to scale without migrating to the MongoDB adapter. A quantitative evaluation of topology choice versus task outcome quality, latency, and cost, and a systematic red-team evaluation of the prompt-injection surface introduced by tool-bearing subagents, are left to future empirical work.

XI. CONCLUSION

The AI–Collective core shows that a multi-agent LLM orchestration system can be built with the same architectural discipline as conventional software systems: strict hexagonal separation between orchestration logic and concrete providers/infrastructure, a small set of well-defined communication topologies rather than unconstrained agent-to-agent messaging, and a memory and middleware model that treats reliability – bounded recursion, partial-failure tolerance, loop detection, cost budgeting – as a first-class, uniformly applied concern. The resulting provider-, storage-, queue-, and lock-agnostic design lets one codebase serve both local experimentation and distributed production deployment.

REFERENCES

- [1] LangChain AI, “LangGraph: Build resilient language agents as graphs,” <https://github.com/langchain-ai/langgraph>, 2024.
- [2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv:2308.08155*, 2023.
- [3] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” *arXiv:2308.00352*, 2023.
- [4] J. Moura et al., “CrewAI: Framework for orchestrating role-playing, autonomous AI agents,” <https://github.com/crewAIInc/crewAI>, 2024.